

Aula 5: Treinando redes II — convergência, superfícies de perda e taxa de aprendizado

Descida de gradiente • Número de condição • Momento

Prof. Marcelo Keese Albertini

GBC073 — Inteligência Computacional • Universidade Federal de Uberlândia

2026



- ▶ Já sabemos **representar**: MLPs aproximam qualquer função contínua
- ▶ Já sabemos **derivar**: retropropagação calcula $\frac{\partial E}{\partial w}$ em tempo linear no tamanho da rede
- ▶ A receita é iterar $w \leftarrow w - \eta \frac{\partial E}{\partial w}$... mas:

Intuição

Ter o gradiente *não* garante chegar a um bom mínimo, nem chegar rápido. A pergunta desta aula é: **quando e com que velocidade a descida de gradiente converge — e o que fazer quando ela não converge?**

- ▶ Hoje: superfície de perda, análise de convergência, taxa de aprendizado, *schedules* e momento
- ▶ Próxima aula: SGD, minilotes e otimizadores adaptativos (Adam, AdamW)

PARTE 1

A superfície de perda



Definição — Superfície de perda

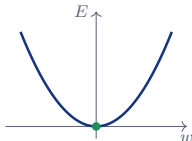
Dado um conjunto de dados $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, a superfície de perda é a função $E(w) = \frac{1}{N} \sum_{i=1}^N \ell(f(x_i; w), y_i)$, vista como função dos pesos $w \in \mathbb{R}^d$. Treinar é resolver $\arg \min_w E(w)$.

- ▶ A entrada da rede é x ; a variável da otimização é w , com d indo de milhões a trilhões
- ▶ Não dá para visualizar — mas dá para **analisar**: a geometria dessa superfície decide velocidade, estabilidade e qualidade do treino

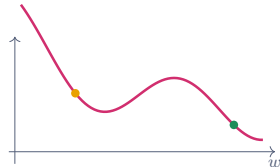
Cuidado

Para redes com camadas ocultas, $E(w)$ **não é convexa**: há simetrias (permutar neurônios não muda a função), platôs, selas e vales curvos. Toda a teoria clássica de otimização convexa vale apenas *localmente*.

Convexa: um único mínimo



Não convexa: mínimos, platôs, selas



Intuição

Visão moderna (Dauphin et al., 2014; Choromanska et al., 2015): em altíssima dimensão, o vilão típico *não* é o mínimo local ruim — são os **pontos de sela** e os **platôs**, onde o gradiente quase some. Em redes largas, quase todos os mínimos locais têm perda parecida com a do mínimo global. Por isso a descida de gradiente funciona na prática.

PARTE 2

Convergência do gradiente descendente



O laboratório perfeito: a quadrática em 1D

Perto de um mínimo, toda superfície suave é aproximadamente quadrática (Taylor)

Tome $E(w) = \frac{1}{2}a w^2$ com $a > 0$ (curvatura $E''(w) = a$). A atualização $w \leftarrow w - \eta E'(w)$ vira:

$$w_{t+1} = w_t - \eta a w_t = \underbrace{(1 - \eta a)}_{\text{fator de contração}} w_t \implies w_t = (1 - \eta a)^t w_0$$

Teorema — Convergência na quadrática

A descida de gradiente converge se, e somente se, $|1 - \eta a| < 1$, isto é, $0 < \eta < 2/a$. Com $\eta = 1/a$ ela converge em *um único passo*.

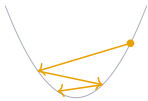
- ▶ $\eta < 1/a$: converge de forma monotônica (mesmo lado do vale)
- ▶ $1/a < \eta < 2/a$: converge *oscilando* de um lado para o outro
- ▶ $\eta > 2/a$: **diverge geometricamente** — a perda explode

Os três regimes, desenhados

$\eta\alpha = 0,5$: **monotônico**



$\eta\alpha = 1,7$: **oscila e converge**



$\eta\alpha = 2,4$: **diverge**



Intuição

A taxa de aprendizado ótima é o *inverso da curvatura*: $\eta^* = 1/E''$. Vales muito curvos pedem passos pequenos; vales rasos pedem passos grandes. Guarde essa frase — ela explica o método de Newton, o RMSProp e o Adam.

Perto do mínimo, $E(\mathbf{w}) \approx \frac{1}{2} \mathbf{w}^\top H \mathbf{w}$, com H a **Hessiana** (curvaturas), autovalores $\lambda_1 \geq \dots \geq \lambda_d > 0$: em cada autodireção, a quadrática 1D com $a = \lambda_i$.

Teorema — Convergência no caso multivariado

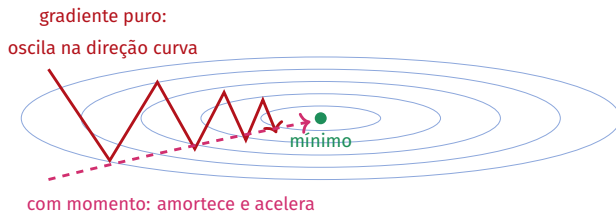
Converge sse $\eta < 2/\lambda_{\max}$: a direção *mais curva* dita o teto da taxa. A velocidade é limitada pela *mais rasa*: com a melhor η fixa, o erro cai como $\left(\frac{\kappa-1}{\kappa+1}\right)^t$.

Definição — Número de condição

$\kappa = \lambda_{\max}/\lambda_{\min}$. Mede a *anisotropia* do vale: $\kappa = 1$ é uma tigela redonda; $\kappa \gg 1$ é um desfiladeiro estreito e comprido.

- Redes mal inicializadas ou sem normalização têm κ enorme — normalização de entrada, *batch norm* e boas inicializações aceleram o treino porque *reduzem* κ

Por que κ grande dói: o ziguezague



- ▶ O gradiente aponta *perpendicular às curvas de nível*, não para o mínimo: na direção curva ele é gigante, na rasa é minúsculo
- ▶ A η precisa caber na direção curva ($\eta < 2/\lambda_{\max}$) — e aí o avanço na direção rasa fica lento demais
- ▶ Duas saídas: **(i)** usar uma taxa *por direção* (Newton, Adam) ou **(ii)** acumular velocidade (**momento**)

Exercício 1 — Regimes de convergência

Considere $E(w) = 2w^2$ (logo $E'(w) = 4w$) com $w_0 = 1$.

- (a) Com $\eta = 0,1$, calcule w_1, w_2, w_3 . O comportamento é monotônico ou oscilante?
- (b) Com $\eta = 0,4$, calcule w_1, w_2, w_3 . O que mudou?
- (c) Com $\eta = 0,6$, calcule w_1, w_2, w_3 . O que acontece com $E(w_t)$?
- (d) Qual η leva ao mínimo em um único passo?

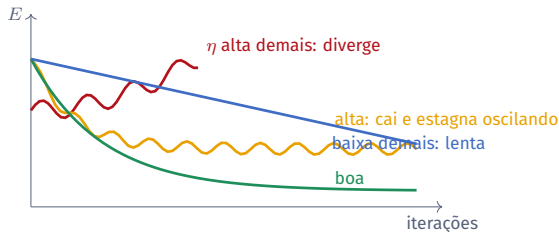
Intuição

Antes de calcular, preveja: aqui a curvatura é $a = 4$, então o teto é $\eta < 2/4 = 0,5$ e o passo perfeito é $\eta = 1/4$.

PARTE 3

Taxa de aprendizado na prática





Cuidado

A perda que **oscila e cresce** não se resolve com mais épocas: é divergência geométrica (η acima do teto $2/\lambda_{\max}$). Reduza a taxa — tipicamente por um fator de 3 a 10 — e recomece.

Taxa fixa é desperdício: *schedules*

O padrão moderno: aquecimento linear + decaimento cosseno

Definição — *Schedule* de taxa de aprendizado

Uma função η_t que varia ao longo do treino. O padrão dominante em redes modernas (incluindo *transformers* e LLMs):

$$\eta_t = \begin{cases} \eta_{\max} \frac{t}{T_w} & t < T_w \quad (\text{aquecimento linear}) \\ \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \pi \frac{t - T_w}{T - T_w} \right) & t \geq T_w \quad (\text{cosseno}) \end{cases}$$

- ▶ **Por que aquecer?** No início a curvatura local é enorme e mal estimada; passos grandes destroem o treino
- ▶ **Por que decair?** No fim, taxa alta fica quicando nas paredes do vale sem afinar a solução
- ▶ Em resumo: η_t grande explora, η_t pequena refina

```
opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)

def aquece_e_decai(t, T_w=500, T=10_000):          # fator multiplicativo sobre lr
    if t < T_w:
        return t / T_w                            # aquecimento linear
    p = (t - T_w) / (T - T_w)                      # progresso em [0, 1]
    return 0.5 * (1 + math.cos(math.pi * p))        # decaimento cosseno

sched = torch.optim.lr_scheduler.LambdaLR(opt, aquece_e_decai)

for x, y in loader:
    opt.zero_grad()                                # PyTorch ACUMULA gradientes: sempre zere antes
    perda = criterio(model(x), y)
    perda.backward()                                # retropropagacao (Aula 4)
    opt.step()                                       #  $w \leftarrow w - lr_t * grad$ 
    sched.step()                                     # avanca o schedule
```

Esquecer o `zero_grad()` soma gradientes de lotes diferentes — um dos bugs mais comuns em treino.

PARTE 4

Momento



Definição — Descida de gradiente com momento

Mantenha uma *velocidade* v que acumula gradientes passados com fator $\beta \in [0, 1)$:

$$v_{t+1} = \beta v_t + \frac{\partial E}{\partial w} \Big|_{w_t}, \quad w_{t+1} = w_t - \eta v_{t+1}$$

Valor típico: $\beta = 0,9$ — cada gradiente ecoa por $\approx 1/(1 - \beta) = 10$ passos.

- ▶ Na direção **rasa**, os gradientes apontam sempre para o mesmo lado: as contribuições se *somam* e o passo efetivo cresce até $\approx \eta/(1 - \beta)$
- ▶ Na direção **curva**, os gradientes alternam de sinal: as contribuições se *cancelam* e a oscilação é amortecida
- ▶ Resultado: o ziguezague vira uma trajetória suave vale abaixo

Teorema — Aceleração em vales mal condicionados

Em quadráticas, o gradiente puro precisa de $O(\kappa)$ iterações por fator de redução do erro; com momento bem ajustado, $O(\sqrt{\kappa})$. Para $\kappa = 10^4$: dez mil passos vs. cem.

Definição — Momento de Nesterov (1983)

Avalie o gradiente no ponto *para onde a inércia já está levando*:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \frac{\partial E}{\partial \mathbf{w}} \Big|_{\mathbf{w}_t - \eta \beta \mathbf{v}_t}$$

O “olhar adiante” corrige a velocidade antes de exagerar.

Momento **não** é acessório: SGD com momento 0,9 + cosseno segue competitivo com o Adam em visão — muitas vezes generalizando melhor.

Implementação manual — exatamente a convenção interna do PyTorch:

```
v = [torch.zeros_like(p) for p in model.parameters()]

with torch.no_grad():
    for p, vp in zip(model.parameters(), v):
        vp.mul_(beta).add_(p.grad)      # v <- beta*v + grad
        p.add_(vp, alpha=-lr)          # w <- w - lr*v
```

Equivalente pronto (e a versão de Nesterov):

```
opt = torch.optim.SGD(model.parameters(), lr=0.1,
                       momentum=0.9)      # Polyak
opt = torch.optim.SGD(model.parameters(), lr=0.1,
                       momentum=0.9, nesterov=True)  # Nesterov
```

Alguns livros usam $v \leftarrow \beta v - \eta \nabla E$; $w \leftarrow w + v$. Mesma dinâmica, hiperparâmetros *não* intercambiáveis — confira a convenção do artigo.

PARTE 5

Panorama e curiosidades



Para discutir em aula

Durante o treino, a perda caiu bem por 2 000 iterações e depois passou a oscilar sem cair, num patamar alto. Um colega sugere: “deixa treinando a noite toda, com mais iterações ela desce”.

A sugestão resolve / não resolve — e por quê?

Para discutir em aula

Durante o treino, a perda caiu bem por 2 000 iterações e depois passou a oscilar sem cair, num patamar alto. Um colega sugere: “deixa treinando a noite toda, com mais iterações ela desce”.

A sugestão resolve / não resolve — e por quê?

Fechamento

Não resolve. Oscilar num patamar é o regime $1/\lambda < \eta < 2/\lambda$ das direções curvas: o ponto fica quicando entre as paredes do vale, e mais iterações só repetem o quique. O remédio é **reduzir η** (manualmente ou com um *schedule* de decaimento) — e é exatamente por isso que o decaimento cosseno derruba a perda no fim do treino.

- ▶ **Newton:** $w \leftarrow w - H^{-1} \nabla E$, isto é, $\eta = 1/\text{curvatura}$ por direção. Perfeito em teoria; inviável com d na casa dos bilhões (H tem d^2 entradas)
- ▶ **Quickprop (1988) e Rprop (1993):** passo *por peso* ajustado por sinais de gradientes — avós diretos do RMSProp e do Adam, que veremos na próxima aula
- ▶ **Otimização sem gradiente** (algoritmos genéticos, estratégias evolutivas): dispensa retropropagação, mas escala mal com d ; hoje vive em nichos (busca de arquiteturas, recompensas não diferenciáveis). Voltaremos a ela só como panorama, no fim do curso

Intuição

A linha mestra do deep learning moderno é clara: **tudo que puder ser diferenciável, será** — porque o gradiente é a única informação de busca que escala para trilhões de parâmetros.

- ▶ A superfície de perda de uma rede profunda é **não convexa**, mas em alta dimensão os vilões práticos são selas e platôs — não mínimos locais ruins
- ▶ Na quadrática, tudo se decide por η vs. curvatura: converge sse $\eta < 2/\lambda_{\max}$; passo ótimo $\eta = 1/\lambda$
- ▶ O **número de condição** κ mede a anisotropia do vale e dita a velocidade: $O(\kappa)$ para gradiente puro, $O(\sqrt{\kappa})$ com momento
- ▶ Na prática: diagnostique pela curva de perda; use **aquecimento + decaimento cosseno**; SGD com momento 0,9 é uma base forte até hoje
- ▶ Próxima aula: por que treinar com *minilotes* (SGD), e os otimizadores adaptativos — RMSProp, Adam e AdamW